

弊社生産構造の理解要約（ChatGPT理解確認用）

1. 製品と構成

- 完成品から中間品・部品まで段階的な構成を持つ多層BOM（段階BOM）を採用。
- 各段階（サブ1、サブ2、コンプ、気密、艀装）はそれぞれ製品コード（WIP）を持ち、必要に応じて在庫（または見なし組立）として扱う。
- 中間品を在庫化しない段は `is_phantom=1` で区別。

構造例：

```
P_FINAL → P_OUTFIT → P_AIR → P_COMP → P_SUB2 → P_SUB1
```

2. ラインと工程

階層	時間単位	管理目的
ライン	日単位	粗い計画（負荷・納期感）
工程（ライン配下）	分単位	詳細計画・実行スケジュール

- ラインのLTは製品ではなく **ルーティング工程（`m_routing_step`）** に紐づく。
- ライン既定LTは `m_line_default`、製品別例外は `m_line_product_lt`。
- 実際のスケジュールは `m_routing_step` に展開して使用。

3. セル工程（逐次流し）

- コンベアではなく**セル生産方式**。同一ロットを複数工程で段階的に処理。
- 前工程で一定数量（移送バッチ）が完成すると次工程が着手。

項目	意味	保持先
サイクルタイム	製品×工程×ラインの秒/個	<code>m_cycle_time</code>
段取り時間	工程ごとのセットアップ時間	<code>m_cycle_time</code>
ロットサイズ	例：40台	<code>m_routing_step_param.lot_size</code>
移送バッチ	例：4台	<code>m_routing_step_param.transfer_batch_qty</code>
着手条件	例：バッチ到達で着手	<code>m_routing_step_param.start_trigger</code>
1日時間枠	例：240分（4時間）	<code>m_routing_step_param.daily_time_window_min</code>
バッファ	安全在庫（8～20台）	<code>target_buffer_qty, max_buffer_qty</code>

所要時間 = 段取り + サイクル × 数量 / 60（分単位）

4. リードタイムの三層構造

種類	内容	保持先
製品全体LT	受注～出荷の目安	<code>m_product.standard_lt_days</code>
構成内LT	親内オフセット	<code>m_product.self_lt_days</code>
工程LT	スケジュール分布	<code>m_routing_step.lead_time_days / duration_min</code>

- 日単位（粗計画）と分単位（詳細計画）を併用。
- 工程LTを積み上げて製品LTを導出可能。

5. 調達区分とLT

区分	供給源	LT決定方法
MAKE（自社製造）	自工程	下位品目のルーティングから逆算
BUY（購買）	仕入先	<code>m_supplier_item.purchase_lt_days</code>
SUBCON（外注）	外注先	同上、または固定LT

- 調達区分は `m_bom_item.sourcing_type` で管理。
- 仕入先は `m_supplier`、購買LTは `m_supplier_item` に登録。

6. スケジューリング（基本ロジック）

1. 受注数量・納期を入力。
2. ルーティングを降順に読み込み。
3. 各工程で：
 - 日単位LT → `lead_time_days` で日移動。
 - 分単位 → `duration_min` または `m_cycle_time` から算出。
4. カレンダ（`m_calendar_day.work_minutes`）で日跨ぎ補正。
5. 移送バッチ・バッファ条件で前後工程を逐次制御。
6. 購買品は投入予定日から `purchase_lt_days` を逆算して発注日を計算。

7. 全体像（データ構造と運用思想）

- 段階BOM+ルーティング+サイクル+パラメータの4軸構造。
- ラインと工程を日と分で分離して現実的な計画を実現。
- セル生産・ロット・バッチ・バッファの挙動をデータ化。
- 購買・外注を統合して納期逆算が可能。

8. 一文でまとめると

御社は「日単位のライン計画と分単位のセル工程を組み合わせた多段階組立生産」を行っており、中間品コード・サイクルタイム・移送バッチ・バッファを持つことで、現実の仕掛と段取りを忠実にデジタル化できる構造を採用している。